CS62: Final Project

The final project will be an open-ended software development project done in **groups of 3-4** over the course of 5 weeks. Your group will apply methods in human-centered design to identify a problem that software may be able to solve in the real world. Your software project will likely fall into either one of two camps: interactive or data analysis. You will decide on the best data representations and custom data structures to solve your problem. You will make a public Github repo with a detailed README with usage examples for your software project. Lastly, you will create a PDF report that includes run time and affordance analysis for your software project.

The first 1-2 weeks of the project will be dedicated to needfinding and figuring out/scoping out your project and finding the data set. You will then have 3 weeks to implement your project.

The project is purposefully open-ended, which may be daunting, especially if you do not have prior experience in large unstructured software applications before. The instructors are here to help and give constraints and direction! The project is contract graded, which means you decide what grade you get as long as your group completes the promised work correctly by the deadline.

Learning goals

- Apply the human-centered design process to software engineering
- Apply what you have learned to select (and customize and create) the most appropriate data structure for a problem
- Judge the appropriate scope of problems that can and can't be solved with computer science
- Analyze user-created algorithms for efficiency (time/space complexity), edge cases, and affordances
- Develop and format an open-source Github repo to be readable and usable for the public

Timeline

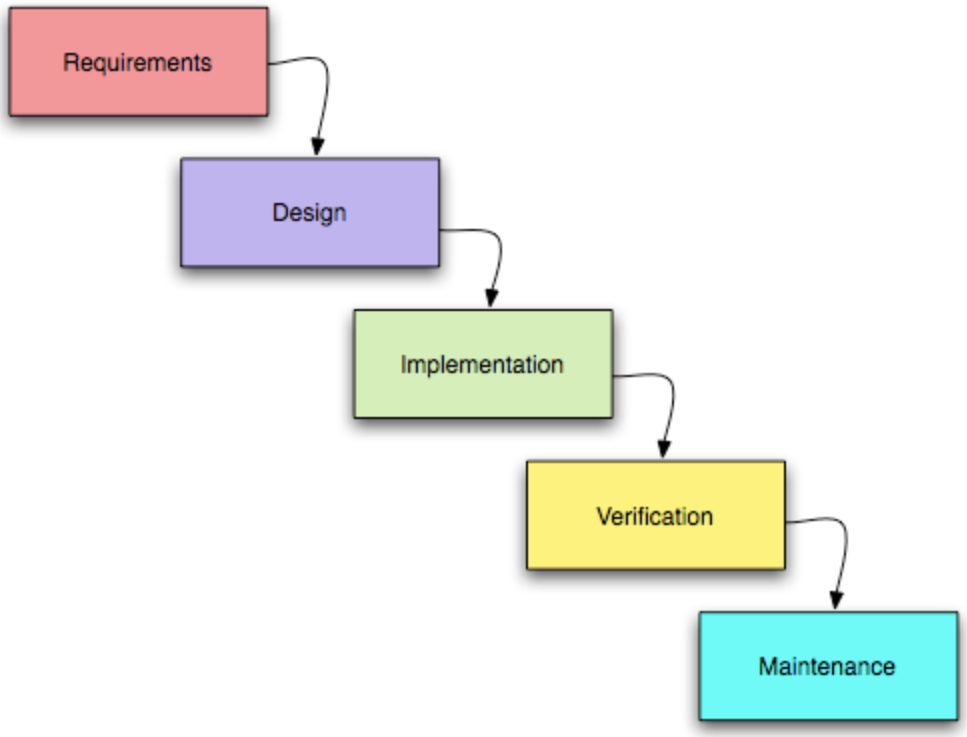
- Part 0: Check-in during lab Weds April 15 (or before)
  - Problem specification & data structure proposal
  - Data format/generation proposal
- Part I: Due Weds April 22th 11:59pm

    - Final problem specification & data structure proposal
    - Generated data
    - Grading contract
- Midpoint check-in during lab April 29
- Part II: Due Weds May 13th 11:59pm
    - Final Github repo, code, README
    - Final PDF report
    - Self & peer evaluation

If you have seniors in your final project group, Part II is due May 6th due to the senior grade deadline.

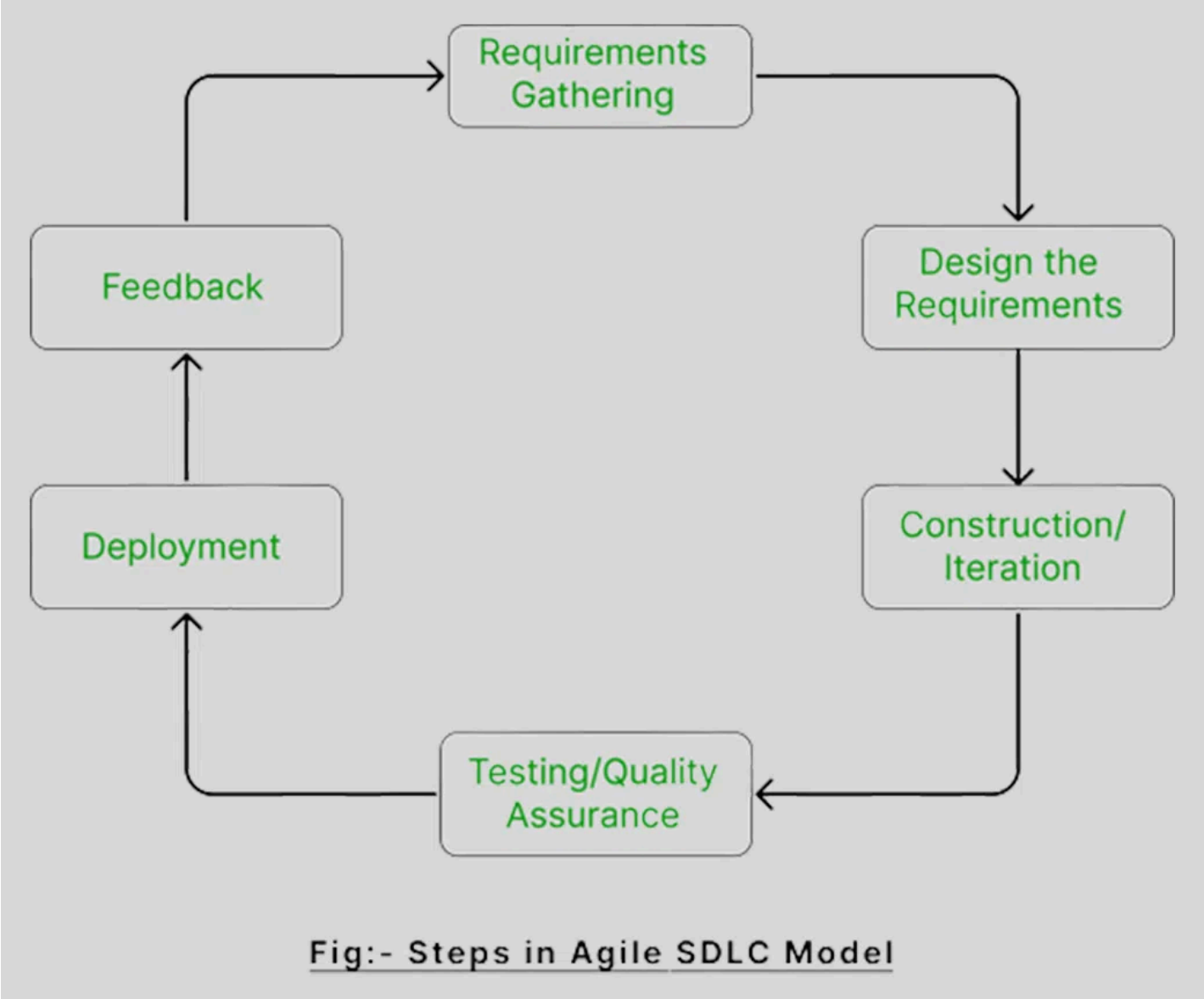
Background: Human-Centered Design

Traditionally, large software engineering projects have been managed with the "waterfall" method, where engineers figure out software requirements first, which inform their design, which informs their implementation, which informs their testing, deployment, and maintenance. This process is not really iterative: after specing a piece of software at a high level, you assume this is how it's going to be going forward. (Just take a look at old government software or the my.pomona.edu portal: my guess it was designed only once and rarely iterated upon, much to users' chagrin.)



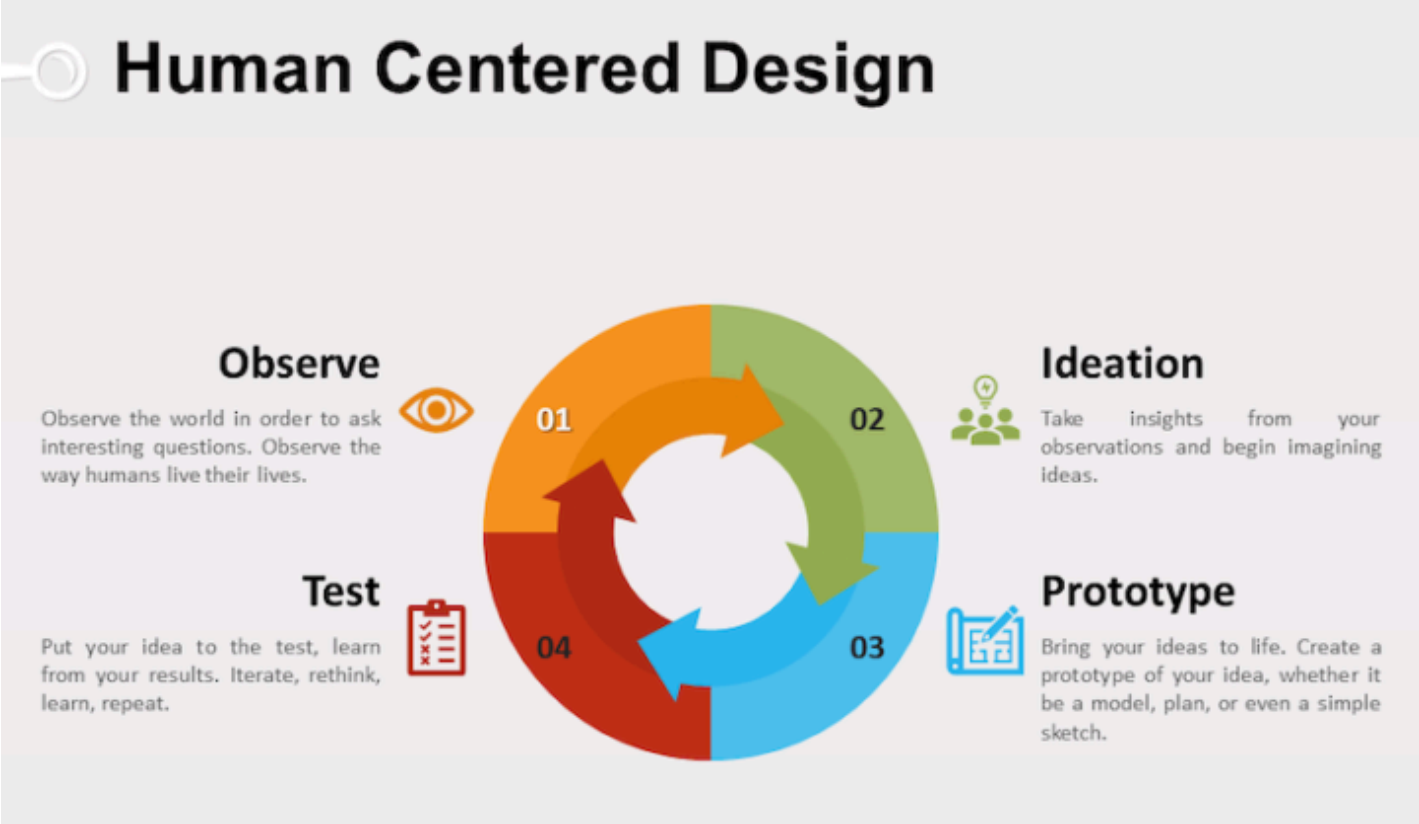
The traditional "waterfall" method of software engineering used in the early 80s (via)

More recently, modern companies largely use the “agile” method of software development. The agile method takes lessons from *iterative* feedback loops of human-centered design to incorporate user feedback in software engineering. The focus, however, is more on developing software quickly: engineers go through one iteration of this loop often in a month-long “scrum” session.



The more recent "agile" method of software engineering (via)

Human-centered design, on the other hand, (also known as user-centered design), places the emphasis on the *end user* of the software, not just the software requirements or specifications.



One version of the human centered design process: an iterative loop that centers user needs. (via)

For this project, you will be doing a modified version of the human-centered design loop. You will not be evaluating or iterating on your software project due to time constraints, but you will be conducting user interviews for need finding (the observe & ideation stages) that inform your design and prototyping of your software.

Before getting into the detailed steps, here are two sample projects. I imagine your projects will fall into one of two categories: interactive software (interaction from reading user input from main() is fine) or data analysis. If you feel like your proposal falls in a different category, that's totally fine, just check with the instructor.

Examples

Interactive software project example: Synced playlists

Bella makes playlists they want to share with their friends, but they use Spotify while friend A uses Apple Music and friend B uses YouTube Music. While interviewing their friends and asking for stories about their music sharing experiences, they learned about turntable.fm, a (now-defunct) website that let users take turns live DJing and queuing songs for each other. Inspired by this anecdote, but still wanting asynchronous, any time music sharing in the form of playlists, Bella decides to build a Java app that will take as input a text file of Spotify or Apple Music or YouTube Music links, allow

users to re-order songs in their app, and generates text files of the playlist in all 3 music sharing formats.

(Bella originally wanted to take as input just a single link to a playlist to read the songs from, but realized the music sharing services' APIs were difficult to hook up with Java. However, if your group wanted to take on a challenge like that, it would be grounds for an A+ on the project.)

First, Bella's group writes code to automatically extract song information (like Title, Artist, Album) from a given link using the Jsoup library. They use this library to also find the URLs on other music providers through parsing their search query (e.g., once given a Spotify link like https://open.spotify.com/track/6LgJvl0Xdtc73RJ1mmpotq, they scrape the title and artist to be Paranoid Android by Radiohead. They then can format a YouTube search URL like https://www.youtube.com/results?search_query=paranoid+android+radiohead and scrape the first result to get the URL to the YouTube video corresponding to the song).

Bella's group creates a custom Song data structure to contain all 3 links, as well as song metadata. They create a custom equals() and hash function which only looks at song metadata instead of the URLs. However, they realize the best data structure to store their songs in is not a hashtable but a doubly linked list, since the songs need to be ordered as a playlist. Their DLL supports moving songs around, and they also use a queue to implement the "add to queue" feature that asks the user for a new song URL which will output the song before the rest of the playlist.

The interactive main() of their Java app prompts users to input the name of a .txt file containing song URLs in playlist order. It then parses and prints the URLs of the other streaming services. Users can use their keyboards to order songs, add new songs to the queue, or save output.txt files for the other streaming services.

Data analysis project example: Group personal finance

Alex, David, and Marina like comparing how much money they spent at the end of the month on their group chat. They wonder: why not just build a shared group finance Java app, that takes as input spreadsheets each of them download from their bank, and outputs some data analysis (e.g., who spent the most on eating out this month? How far apart was each of their spending from each other in each category? If the group set a budget to collectively spend less than $100 on entertainment, did they make it? What was each person's top 3 spend categories, and did it differ between them?)

From their interviews where they asked their friends to share and reflect on their past month's spending, they learn that some friends don't track their spending at all (and wished they did, as they found recurring subscription services they don't use), and some friends wanted to share with other friends but were too shy to overcome the social barrier of talking about finances, while some friends

wanted bragging rights on who spent the least amount of money in their friend group. From these interviews, they also decided to implement a leaderboard feature and hone in on the specific kinds of data analysis their software project will do.

The group realizes each of their banks has slightly different categories for spending, so they first write a Java file to automatically clean their data so all the categories are standardized (e.g., one person's spreadsheet has the heading "Restaurants" while another has the heading "Dining Out", but these mean the same thing).

The group implements their app by using a custom data structure to store transactions. The Transaction Class has the spender's name, the name of the merchant, the date of the transaction, the transaction category, and the price. They use a custom Binary Search Tree that allows duplicate keys to store all transactions, where the price is the key (so the BST can have multiple $19.99 transactions, for instance). They debate also storing the Transaction objects in person-specific hashtables, so it is faster to access the Transactions for a single person as opposed to having to filter through searching the whole tree–it is storing redundant data, but they decide the trade off is worth it since many of their data analysis features are person-specific. They decide to use Java's built in mergesort to sort their data.

The group writes in their README what folder to put the spreadsheets in so their Java app can automatically read and clean them. When running the main() method, their software prompts the user to enter a number or letter on their keyboard corresponding to what analysis they want: for instance, pressing L shows the leaderboard of overall lowest spending per person, while pressing R shows each person's spending on restaurants.

Past classmates' projects

Sample projects from previous semesters include…

- Displaying the 5C menu items and including macros, to create a customized meal plan that met specific targets (like what should I eat if I want 50g of protein today)
- Analyzing Instagram DM data for statistical trends like day of week/time of day most messages were sent, or most common words
- A better Hyperscheduler that checked if you met course pre-requisites before choosing classes
- Creating a database of 5C mutual aid resources, making it searchable based on user-specified parameters
- Analyzing Flex data transactions per retailer or per time of day
- BeMusic, like BeReal but for a single song of the day

- Better pantry management, reminding you to cook foods before they expire, or suggesting grocery lists for new recipes

These examples hopefully illustrate the scope that is expected for this final project. Depending on your domain, the data cleaning part may be easy (download existing spreadsheets and standardize them) or hard (use complex APIs to extract song information). One of the purposes of the Part 0 check-in is to make sure getting the data for your project is doable.

Part I: Problem specification

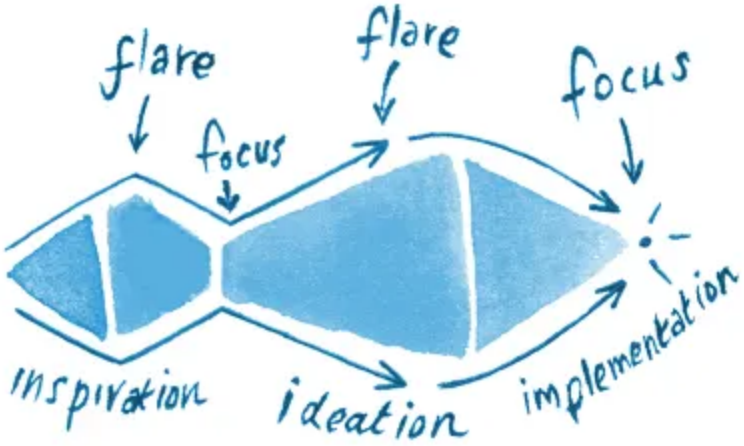
I.A: Needfinding

First, find 30 minutes to hold an initial meeting with your group (it could be after you finish lab, for instance). Maybe you already have specific software projects in mind. Maybe you have no clue. Either way, to situate your exploration of the current state of the world, spend about 10 minutes freely brainstorming either/both:

- Daily problems you or your friends encounter that could have a software solution
- Questions you've always wanted data-driven answers to

One suggestion would be to silently write down as many ideas as possible on a post-it or sheet of paper for 3 minutes, then pass the post-its and iterate on each other's ideas. Generating individual ideas and then giving "yes, and…" feedback or riffing or refining ideas with others can result in more ideas.

If you're really stuck, you can take a look at the data structures we've covered in class and read the history textbook to get examples of applications that these data structures would be good for. (I do not recommend this method for real life design work—do not constrain yourself to the technology, or let the technology dictate what is possible. That's the opposite of human-centered design!)



The "double diamond" of design: flaring to generate many ideas, focusing to converge on ideas (via)

This first step was a "flare" step: generate many ideas and find inspiration. You will be conducting semi-structured interviews to further generate more insights and inspiration. Semi-structured means there's an interview script, but you can go off script and ask follow up questions as the opportunity arises.

Next, spend 20 minutes with your group coming up with *interview questions* to further explore promising initial ideas. Each group member should interview at least one other person who is not in this class (e.g., a friend, a colleague, someone who would be most relevant for the domain you're thinking of). Interviews should take 20-30 minutes and the interviewer should either audio record the interview (to take notes on later) or take notes during it.

The purpose of the interview is to find out more "real life" problems around your proposed domain of software intervention and uncover potentially hidden assumptions. Your interview is not validating if your idea (if you already have one) is a good idea or not, but rather collecting more evidence of activities, problems, and frustrations that are happening in your domain. **You can generate abstractions (i.e., that inform your software specifications) from details, but you cannot generate details from abstractions, so try to collect detailed anecdotes from interviewees.** Some tips:

- Ask a specific question ("tell me about a time when you…") to start things off. It may be tempting to ask a more general question to break the ice, but that may bias the participant at talking at a high level of abstraction for the rest of the interview.
- Get participants to be as specific as possible, i.e., ask them for stories or anecdotes rather than their general feelings.
- Avoid asking yes/no questions. Ask open-ended questions.

- What kinds of participant data can help you motivate and justify your software project? What questions or unknowns do you have around your idea, or around what your project should do?
- For a 20-30 minute interview, 8-10 interview questions should be sufficient. They should be ordered in the order you will ask them. You may also choose to mark questions as skippable or not (do you want an answer to this question for everyone, or is it better to get the juicy details of some tangent the participant went on?).
- After the interview, before leaving, immediately jot down 3 surprising impressions or takeaways.

After everyone has conducted their interview, meet again as a group. Now is time for the "focus" part of brainstorming: focus your inspirations and ideas into a single project to tackle. Discuss the results and high level takeaways of each interview. During this meeting, take notes on:

- What were problems uncovered that might not be best for software solutions? (What other avenues would you recommend instead, e.g., behavioral change? structural change? new norms? Better UI/UX (which is less an engineering problem and more a design problem?))
- What were problems uncovered that you *do* think could be solved with software? Why is software a good match?

Note that these can (and should) be sub-problems: you have only 3 weeks to actually build your proposal. Make simplifying assumptions so your project is in-scope for this class, but justify all your assumptions. For instance, if your found problem is that you want to keep track of books checked out on Little Free Libraries across Claremont, when they get removed, when they get returned, and create user profiles to offer recommended books' locations, it seems hard to track when books get checked out/returned in real time without physically monitoring the locations every day. Why not work on a digital database that lets users browse the books you catalogued ("frozen" at a specific point in time) from the comfort of their computer, letting them traverse a weighted graph with vertices representing the libraries and edges representing the paths between libraries?

A write up of your needfinding results should be included in your final PDF report.

Note: it's fine to pivot your idea after you've done your interviews, but you should include a narrative of why in your PDF report.

I.B: Data formatting & generation

Now that you have your project proposal with what problem you are going to address and potentially some features you want to build, it's time to figure out how to get data for your program. Is data already available to be downloaded, e.g. from an internet database? Do you need to manually generate the data, e.g. through video annotation? Does it make sense to ask a LLM to generate a synthetic dataset to supplement your hand-made dataset for demonstration purposes?

Think about how you will get data for your project. A reasonable dataset should have at least 1,000 entries to test how your data structures scale. If you are having trouble getting to that scale, let's discuss during the Part 0 check-in or on Slack.

A description of how you obtained your dataset should be in the final PDF report.

I.C: Data structure proposal

What data structures will you use to support your project? Will you use the built in Java ones from the Collections Framework, or modify them for your purposes, or create new ones? How do your desired features map to methods of the data structure? Is your data structure the fastest available one possible? Why? What are the drawbacks to using your data structures? If you're using algorithms to process your data, how do the algorithms interface with the data structures?

You are also able to use data structures/algorithms we did not cover in class, as long as you self-study and understand them.

Create a Java Interface file detailing all of your custom data structures. Remember an Interface details what methods should exist, but does not concern itself with the implementation of the methods.

A short written justification of how you chose your data structure should be in the final PDF report.

I.D: Grading contract

The final project is contract graded, which means your group will write a set of criteria to meet in order to get a C, B, or A in the project. (I provide scaffolds which you may fill out.) All grade buckets are centered at the median (e.g., 75, 85, 95), but within the grading buckets, the professor has discretion for the exact point value. You may also propose "bonus" features to ensure an A+ (99-100) on the project. You can think of "features" as distinct user-facing methods—adding songs to a queue is a feature; rearranging playlist order is a feature; converting between Spotify, Apple Music, and YouTube links is a feature. Seeing a leaderboard of each person's spending overall is a feature; seeing a leaderboard of spending per category is a feature (though two different categories do not count as two different features); showing statistics across time (your group spent $100 less on groceries this month than last) is a feature.

- For a C (to pass the final project):
  - Completed PDF write up (including needfinding results and algorithmic & affordance analysis)
  - Valid dataset file with 10-999 entries
  - Github Repo with a README on how to run the code

    - At least 1 data structure used and at least 1 working feature (**TODO:** Describe what the data structure is and what the feature will be (likely your project's most important feature))
- For a B:
    - What is required for a C, and
    - Valid dataset file with >= 1000 entries
    - Github Repo that details your custom data structure's public API
    - At least 2 working features (**TODO:** What features?)
- For an A:
    - What is required for a B, and
    - Github Repo that details usage examples
    - Your code has good style, JavaDocs, and comments
    - At least 3 working features (**TODO:** What features?)
- For an A+
    - What is required for an A, and
    - 1-2 "extra" options that may be outside the scope of the class, such as reading external APIs, creating a graphical user interface

Feel free to modify or propose additional points specific to your project outside of my given scaffolds; this serves as a suggested starting point.

Part 0 check-in requirements

Every group is required to complete a part 0 check-in before turning in Part I. While we will have time in lab on April 15 to do this check-in, I encourage your group (or at least 1 member of the group) to stop by office hours for an earlier check-in so you can feel confident in what you turn in for Part I. Please prepare a short (<5 min) slide-based presentation for the instructors that details:

- The problem you want to solve and if your project is going to be more interactive or data analysis
- Why you decided on that problem (eg from personal experience, from interviews)
- The proposed format for your data & how you plan on generating the data
- Initial thoughts about what data structures you will use
- Any questions you have/advice you want going forward

Note that the grading contract and the actual generated data are not due during the Part 0 check-in, but you are welcome to present either for feedback.

Part I assignment turn in

By Weds April 22nd 11:59pm on Gradescope, please turn in the following files:

- Your dataset (could be a .csv, or some other format)
- Your .java Interface file proposing your data structure
- A PDF that includes:
  - A paragraph about what problem your project solves and what features you anticipate building
  - A paragraph about the results of your need-finding and how it informed your project choice
  - Your grading contract
  - A link to (or embedded) your Part 0 check-in slides
  - Any thoughts, worries, or feedback about part 1

You will be assigned a bucket grade (check plus, check, check minus) as a calibration metric for your project/ideas, but this **does not affect your final grade in any way**. The point of this assignment is to get feedback on your idea and feasibility. Please make sure the submitting person includes their group members on the submission.

Part II: Execution

Now that you've planned out your project, it's time to actually implement what you propose! Here are some tips:

- Work off of a smaller dataset (for instance, the first 10 rows) to test your data structures
- Thoroughly test one data structure or feature before moving onto another that builds off of it
- Plan and abstract your code so each feature is a public method that you can call in main() to test. After all the functionality ("backend") is done, you can worry about taking user keyboard inputs to call your methods interactively ("frontend").

Part II.A: Software development

Please put your files in an appropriately named package. Your Github README should clearly mark which Java file to run the main(). You may also choose to take command line arguments instead; this is fine, as long as you specify in the README.

Remember your `Class`es should now `implement` the interface you defined in Part I! It's totally OK for specifications to change between Parts I and II as well.

We are relaxing the rules of LLM usage for this final project: if we have not gone over something in class that you find you need to use to get your final project done (e.g., reading external libraries or creating a GUI), it is fine to ask ChatGPT for help in generating and explaining sample code for said library. **All LLM generated code and help must be credited in header files and include a link to the conversation**. If you use LLM generated code on something that we did learn in class, your project will be docked points. If you use help from an LLM but it is uncredited, this is a serious violation of our academic honesty policy. Please ask the instructor for cases you are unsure about.

Part II.B: Github

Your code should be hosted in a public Github repository with a README.md (both of which you make). Each group member should be committing and contributing to the repository. The README should include:

- At minimum, how to run the code
  - If you are using external libraries, place the .jar files in a /lib folder and also commit them. Check out the Project Manager for Java VSCode extension to be able to import their classes and methods. If any external libraries need to be installed beyond their .jar files, please detail how to do so in the README.
- Instructions on how to call your public methods. That is, you are now writing an API in your README. This could be the same thing as your JavaDocs: What is the method called? What are its inputs and outputs? What is a verbal description of what it does? Don't forget about constructors!
- For each of your public methods, also include usage examples. What do expected inputs and outputs look like in practice? For example, a usage example for `add(x,y)` would be `add(2, 3) = 5`.
- A one paragraph summary in the beginning on what your software does, ideally showing screenshots or examples of your code running.
- Here is a guide on how to make and format a README. The formatting language of READMEs is Markdown.

Your repo, in addition to your code, should also contain your dataset file. If the dataset has sensitive information, please make a note of it in your PDF write up and submit it on Gradescope instead.

Please make sure your repository is public. Projects that have a repository I cannot access will receive a 0.

Part II.C: Write up

Finally, create a PDF write up detailing your software project. This should include:

- A cover sheet with your project's title, group members, a general paragraph introducing your project (it can be the same as the one on your Github README), and **a link to the public Github repository**
- A few paragraphs on the results of your needfinding.
  - What was your initial idea?
  - Who did you interview? (We don't need real names, but like, "roommate", "someone on the soccer team"—descriptors relevant to your idea would help.)
  - What were the takeaways you learned from your interviews? How did these takeaways influence what project you ended up making?
  - What problems did you find that are best *not* solved by software?
- A description of your dataset format, why you chose that format, and how you obtained the data
- A short written justification on how you chose your data structure(s) for this project
- A "results" section showing screenshots of your code execution (e.g., print outs in main()) that demonstrate each and every one of your features
- An "analysis" section. For each of your features:
  1. What pre-conditions are necessary for this feature to run? Are there any obvious edge cases?
  2. What is the time complexity? Why?
  3. What is the space complexity? Why?
- Additionally, include a general "affordance analysis" of your software project. Answer:
  1. Who benefits most from this software project? Who (if anyone) is left behind?
  2. How does your software reinforce or challenge norms or power structures?
  3. What ways (if any) may your software be used maliciously–i.e., what actions does your software afford?
  4. Any other ethical reflections you would like to mention
- A reflection section:
  1. How long did this project take the group, overall? Did it align with your expectations?
  2. How was this process? Did you stick with your original idea and proposed data structures, or did you make changes along the way? If you made changes, why?
  3. Any feedback on the final project assignment itself?

Part II.D: Self & Peer evaluation

We want to make sure everyone is contributing equally in a group. Please fill out this Google form to detail your contributions and your teammate's contributions of this project.

Part II assignment turn in

By Weds May 13th 11:59pm on Gradescope, please turn in the following files:

- Your PDF write up as detailed above
- Your final dataset (if it contains sensitive data that can't be included on Github)

There's no need to submit your code files because they will be downloaded from your Github repo that is linked on your PDF write up.

Midpoint check-in during lab April 29

Feedback I got from last semester's students is they wanted more scaffolding and intermediate deadlines to keep them from doing everything the last minute. Thus, during our lab on April 29, every group should be prepared to show their progress, ideally with at least 1 feature already working. Please prepare another <5 minute presentation detailing:

- A demo of your code working (or almost working) for 1 feature
- Any pivots or roadblocks you've run into since you turned in Part I
- Any questions/concerns you have

Grading details

I mentioned that you will get what grade you proposed in your contract, but I have discretion on the exact points inside the bucket. For instance, if the results of your algorithmic analysis or affordance analysis are incorrect, but otherwise your project was complete with 4+ features, you might get a 90 instead of a 95. If you coded 4 features, but one of them is buggy, then you really only completed 3 features, and your group would get something in the high B range. If some features were particularly impressive or compelling, but you only did 2 of them, you may get an 89 instead of an 85.

Additionally, not everyone in the same group is guaranteed to get the same grade. Your personal grade is affected by the results of the self and peer evaluation feedback form. Students who "pull more weight" will get higher grades than students who were reported to not have contributed as much.